FireWatch — Plan de implementación del modelo en AWS

Nov 4, 2026 · @Lukas

Con los tres CSV disponibles todavía no se puede entrenar un modelo, así que el plan parte por los datos. El objetivo es cumplir la Guía 3.1.5 (Aplicación de modelos al proyecto semestral, ITY1102): modelo de ML entrenado, API de inferencia, despliegue en AWS, CI/CD, dashboard, testing, monitoreo y seguridad básica.

Insumos revisados: el repositorio `firewatch-dev`, `dashboard_conaf_202609281623.csv`, `emas_202609281634.csv` y `rtdata_y2026_202609281633.csv`.

Qué hay en los datos

Ninguno de los tres archivos trae historia meteorológica, que es lo que el modelo necesita para aprender.

Archivo	Contenido	Para qué sirve
`dashboard_conaf`	90 días (1 jul – 28 sep 2026) × 16 regiones. Nivel regional, no comunal. `nro_inc_act` es un acumulado de temporada: parte en 0 el 1 de julio y nunca baja	Etiquetas: restando día contra día, 69 de 1.440 días-región tienen ≥1 incendio nuevo (4,8 %). Valparaíso: 9 días positivos, todos de invierno
`rtdata`	Una sola foto horaria (28 sep 2026, 09:00) de 437 estaciones y 12 variables: temperatura, humedad, viento, precipitación, radiación, presión, etc.	No sirve para entrenar (sin historia). Es el *live data* de la etapa de inferencia
`emas`	Catálogo de 535 estaciones con coordenadas y `com_ia_id` (código comunal; Valparaíso empieza con 5). 38 estaciones de Valparaíso tienen datos en `rtdata`	Unir estación → comuna

La guía asigna la ingesta y el almacenamiento al ramo de Gestión de Datos. El primer paso es pedir ahí el export completo de `rtdata` desde el 1 de julio, no una sola hora.

Decisiones a cerrar antes de programar

Tema	Situación	Recomendación
Nube	El EP1 eligió Oracle Cloud; la guía exige AWS (EC2)	AWS, dejando documentado el cambio
Dónde corre la inferencia	El EP1 propuso ONNX ejecutado dentro de Go; la guía dibuja una API de inferencia separada (FastAPI)	FastAPI. ONNX en Go obliga a usar cgo y la librería C de onnxruntime, que choca con las imágenes non-root mínimas; además se mapea uno a uno con el C4 que se evalúa
Opción de despliegue	La guía ofrece VMs separadas, ECS/ECR, Kubernetes con Minikube o Docker Compose	Docker Compose en EC2: el `docker-compose.yml` actual ya levanta todo el stack. Los manifiestos de Kubernetes quedan como evolución documentada

Qué cumple ya el repo y qué falta

La mayor brecha es el modelo entrenado; el resto es en gran parte despliegue de lo que ya existe.

Requisito de la guía	Estado en el repo	Falta
Modelo de ML y dataset	Reglas académicas v1/v2 sobre datos sintéticos	Dataset real, entrenamiento y artefacto
API REST documentada	Gateway más OpenAPI	Servicio de inferencia
Servidor AWS accesible	Solo local, en loopback	EC2, dominio, configuración de producción
CI/CD (build, test, deploy)	Build y test de Go y frontend con SBOM y escaneo	Etapa de deploy y workflows Python
Dashboard	Frontend React y Grafana	Mostrar probabilidad y versión del modelo
Testing	Muy completo en Go	Pruebas del pipeline de entrenamiento y de la inferencia

Requisito de la guía	Estado en el repo	Falta
Monitoreo	Prometheus, siete alertas, Grafana	Métricas de inferencia, chequeo de disponibilidad externo
Autenticación y HTTPS	Keycloak con JWT RS256	HTTPS

Plan por fases

Fase 0 — Dataset de entrenamiento

Con Gestión de Datos, conseguir la historia horaria de `rtdata` de julio a septiembre y subirla, junto con el CSV de CONAF, a un bucket S3 versionado (la premisa de la guía).

- [ ] Agregar el clima por región y día: temperatura máxima, humedad mínima, viento máximo, precipitación acumulada y días desde la última lluvia.
- [ ] Etiqueta: hubo al menos un incendio nuevo ese día en la región, calculado como diferencia del acumulado.
- [ ] Evitar fuga: `nro_inc_act` del mismo día no entra como variable. `nro_inc_ant` y `nro_inc_prom_quint` sí, como contexto histórico.
- [ ] Usar las mismas variables que traerá `rtdata` en producción, para que entrenamiento e inferencia no diverjan.

Plan B si Gestión de Datos no alcanza: clima histórico de reanálisis por región, declarando la diferencia con los datos de inferencia.

Fase 1 — Pipeline de entrenamiento

Contenedor Python efímero en `training/` que lee de S3 y compara regresión logística contra Random Forest, como propuso el EP1.

- [ ] Con 4,8 % de positivos: `class_weight` balanceado y partición temporal (julio–agosto entrena, septiembre evalúa).
- [ ] Métricas: PR-AUC, recall y Brier, no accuracy.
- [ ] Línea base: climatología (`nro_inc_prom_quint`). Solo se registra el modelo si la supera.
- [ ] Salida: `.onnx` (o `.pkl`) más un JSON con variables, métricas y hash de los datos, en `s3://…/models/firewatch-ml-v1/` , con una fila en la tabla `models` existente (`params_hash` = sha256 del artefacto).

Fase 2 — API de inferencia y conexión con `risk`

`services/inference` en FastAPI, que carga la versión del modelo desde S3 al arrancar. Va en la red interna, nunca publicado.

- [ ] Endpoints `POST /predict` (nivel, probabilidad, versión), `/health` y `/metrics`; FastAPI genera la documentación OpenAPI que pide la guía.
- [ ] En `risk`, un nuevo `Classifier` de tipo modelo (`ml-v1`) que llama a la API con timeout. Si falla, falla cerrado y queda registrado; nunca cae en silencio a la regla, porque rompería la trazabilidad existente.
- [ ] Un `Provider` de `rtdata` real que usa `emas` para unir estación → comuna, en reemplazo de la fuente sintética.
- [ ] Activación con el `POST /v1/models/{version}/activate` que ya existe.

Fase 3 — Despliegue de resultados

El camino asíncrono de la guía ya existe: `risk` publica el lote en Postgres y el frontend lo lee.

- [ ] Mostrar la probabilidad y la versión del modelo en el frontend y en Grafana.

Fase 4 — AWS

EC2 con Ubuntu y ~30 GB de disco. Por la cantidad de servicios (Keycloak, RabbitMQ, PostGIS, cuatro servicios Go, inferencia, monitoreo), 8 GB de RAM es más seguro que 4 GB.

- [ ] `docker-compose.prod.yml` donde solo el proxy publica el 443, con Caddy y certificado automático (requiere dominio; un DNS gratuito sirve).
- [ ] Acceso a S3 mediante rol IAM, sin llaves.
- [ ] Security group: 443 abierto, 22 solo a la IP del equipo.
- [ ] Frontend: `frontend/src/config.ts` tiene `localhost:8080`, `localhost:8081` y `localhost:4173` en el código; pasarlos a variables `VITE_*` de build.
- [ ] Keycloak: actualizar `KC_HOSTNAME` y las redirect URIs. Gateway: actualizar `WEB_ORIGIN`.
- [ ] Apagar la instancia al terminar, como exige la guía.

Fase 5 — CI/CD

- [ ] `training.yml`: lint, pytest y entrenamiento de humo con datos fixture.
- [ ] `inference.yml`: pytest y build de la imagen.

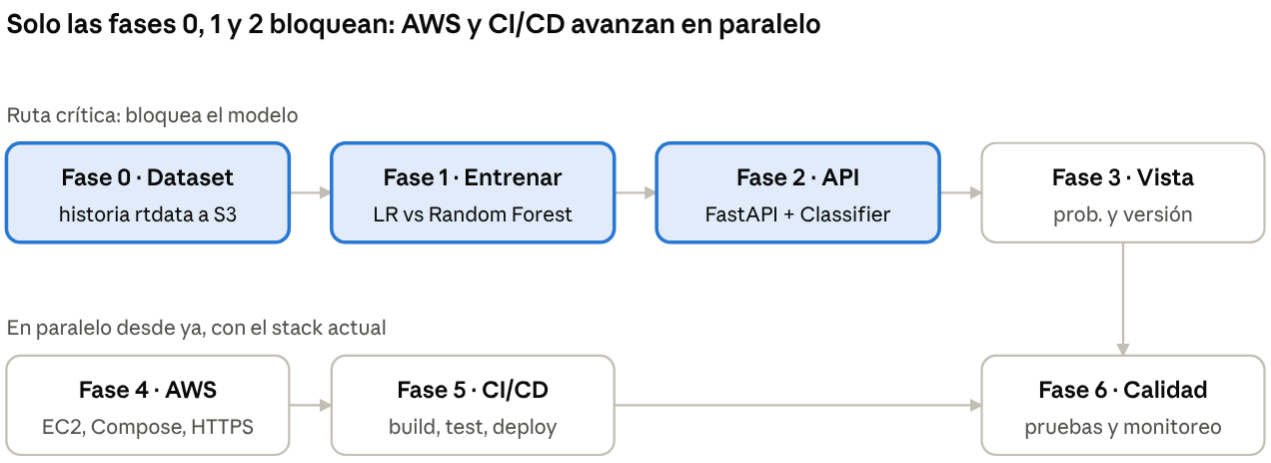
- [ ] `deploy.yml` en `main`: build y test, push a ECR o GHCR, `docker compose pull && up -d` en la EC2 por SSH o SSM, y `scripts/smoke.py` contra la URL pública.

La guía describe reconstruir la imagen con cada modelo nuevo; con el registro existente basta con activar la versión sin rebuild. Ambas opciones son válidas; documentar la elegida.

Fase 6 — Calidad y monitoreo

- [ ] Pruebas unitarias de variables e inferencia, y prueba de contrato entre `risk` e `inference`.
- [ ] Integración con Compose; prueba funcional con `smoke.py` extendido.
- [ ] Métricas de latencia, errores, predicciones por nivel y versión activa.
- [ ] Alarma de disponibilidad externa (CloudWatch sobre el estado de la EC2, o un ping HTTP).

Ruta crítica



ruta crítica · 7 fases, 2 frentes en paralelo

Solo las fases 0, 1 y 2 bloquean el modelo. Desplegar ya en AWS lo que funciona, con HTTPS, saca del camino el riesgo de infraestructura antes de que llegue el modelo.

Advertencia para la presentación. Un modelo entrenado con 90 días de invierno y 69 positivos demuestra el pipeline, no capacidad predictiva; conviene sostener la línea de honestidad que ya tiene la documentación. El modelo comunal para Valparaíso, que es lo que muestra el mapa, requiere el registro histórico de incendios por comuna y fecha, con varias temporadas de verano, y queda como siguiente versión.